Implementation

1. Create database

Create the database and verify that it exists.

```
CREATE DATABASE GAMING_DB;
SHOW DATABASES LIKE 'GAMING_DB';
```

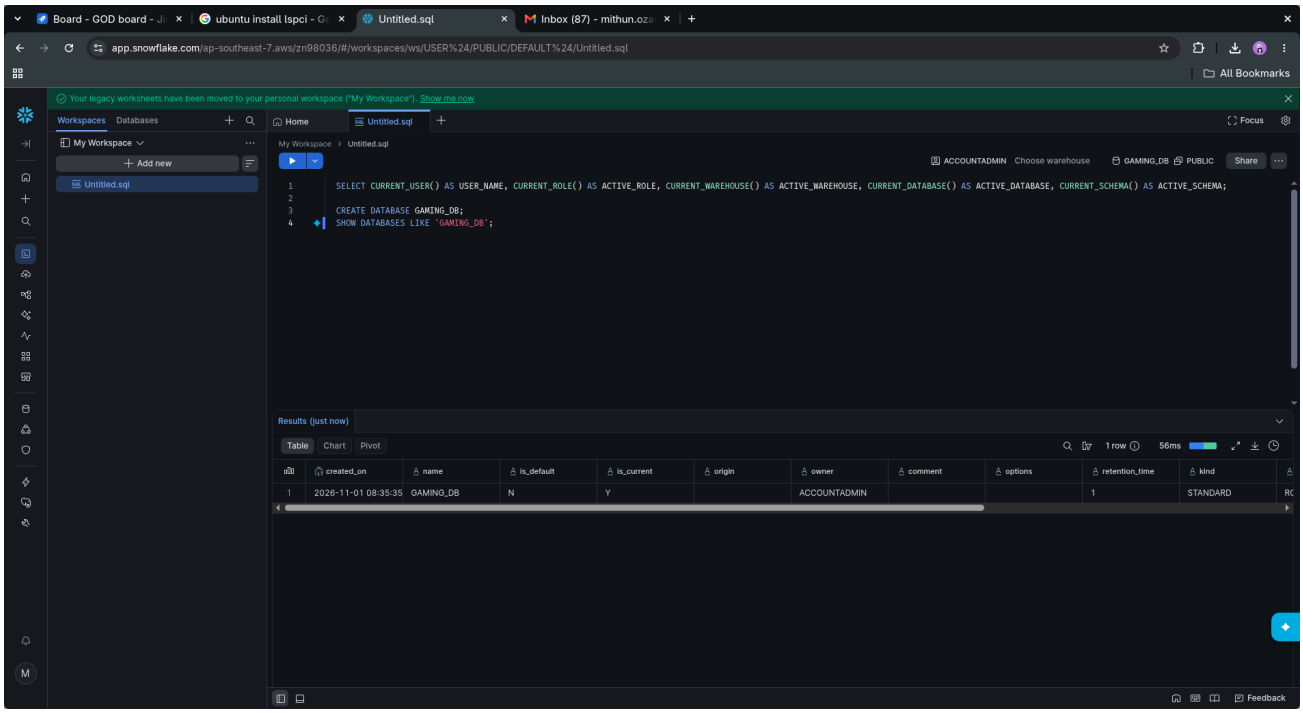


Figure 1. GAMING_DB created successfully.

2. Create schema

```
CREATE SCHEMA IF NOT EXISTS GAMING_DB.SALES;
USE SCHEMA GAMING_DB.SALES;
SHOW SCHEMAS IN DATABASE GAMING_DB;
```

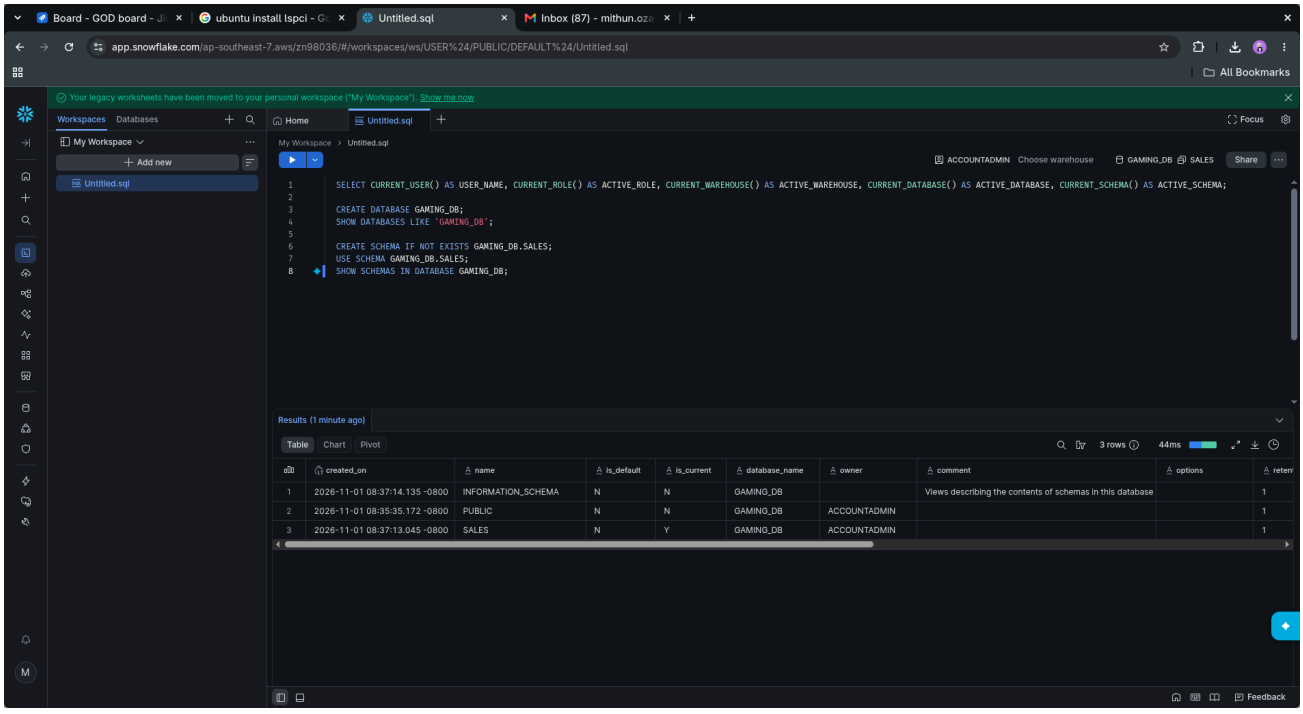


Figure 2. SALES schema created and selected.

3. Create virtual warehouse

```
CREATE WAREHOUSE IF NOT EXISTS GAMING_WH
```

```
    WAREHOUSE_SIZE = 'XSMALL'
    AUTO_SUSPEND = 60
    AUTO_RESUME = TRUE
    INITIALLY_SUSPENDED = TRUE;

USE WAREHOUSE GAMING_WH;
SHOW WAREHOUSES LIKE 'GAMING_WH';
SELECT CURRENT_WAREHOUSE();
```

The warehouse was configured as X-Small, with auto-suspend after 60 seconds, auto-resume enabled and initially suspended.

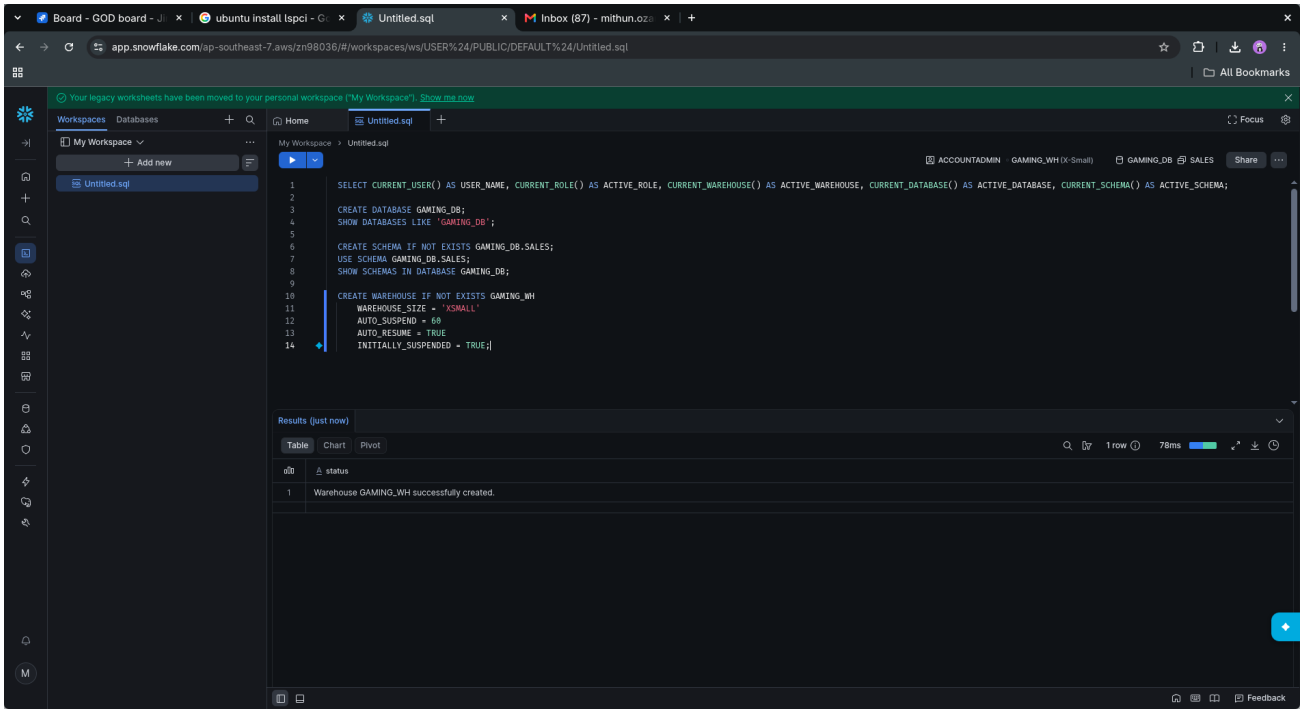


Figure 3. Warehouse created.

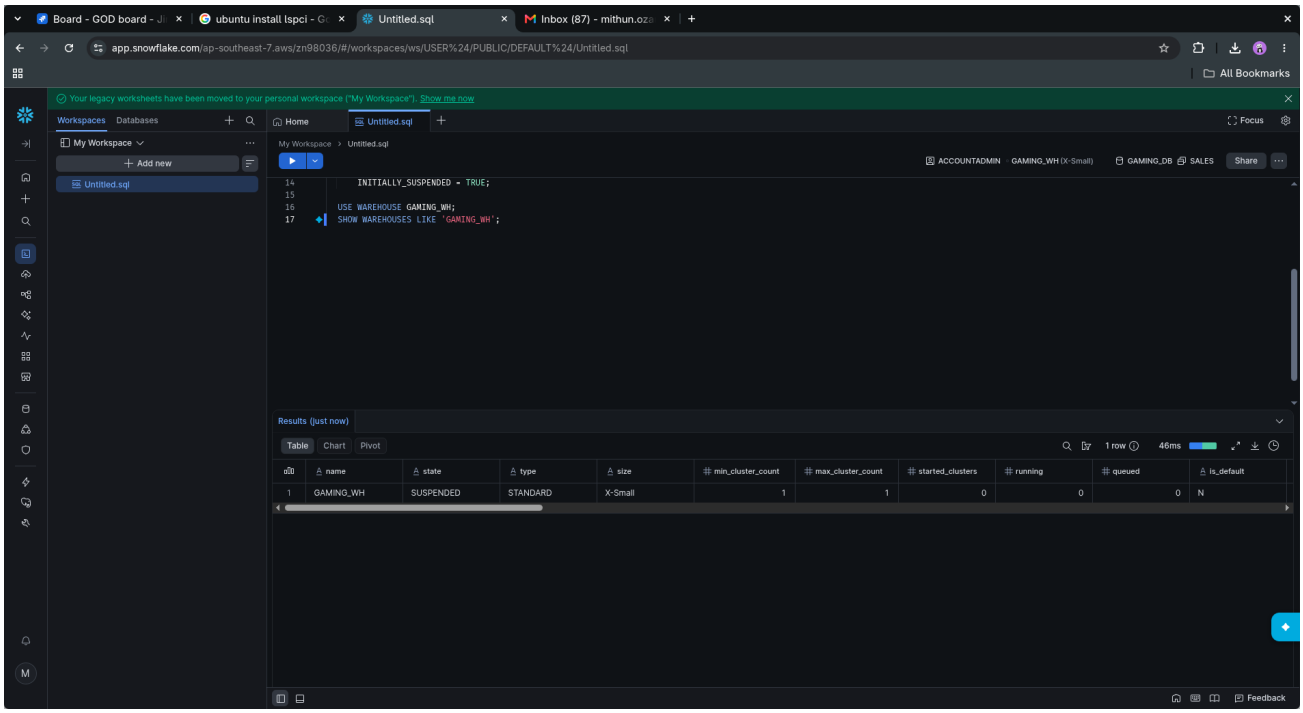


Figure 4. Warehouse shown as X-Small and suspended.

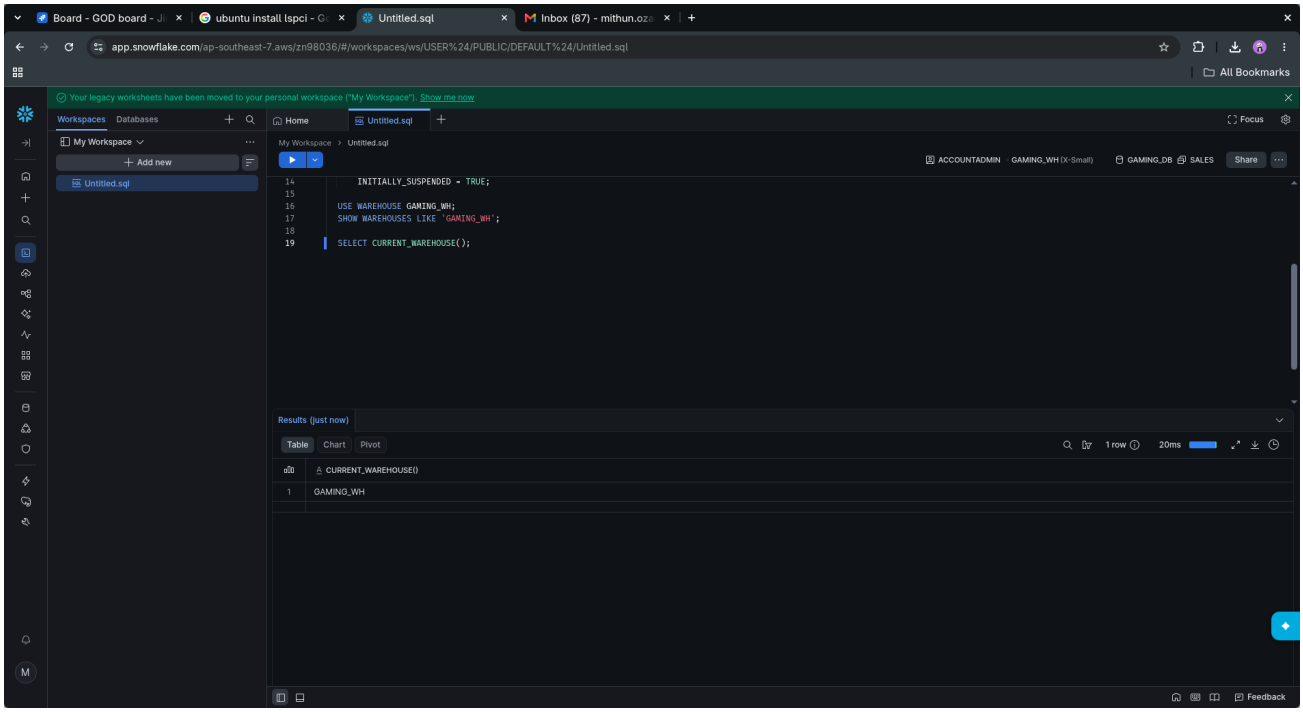


Figure 5. GAMING_WH selected as the current warehouse.

4. Create table

Create GAME_SALES with nine columns to match the CSV file.

```
CREATE TABLE IF NOT EXISTS GAMING_DB.SALES.GAME_SALES (
    SALE_ID NUMBER(10,0),
    SALE_DATE DATE,
    GAME_TITLE VARCHAR(150),
    PLATFORM VARCHAR(50),
    GENRE VARCHAR(50),
    REGION VARCHAR(50),
    UNITS_SOLD NUMBER(10,0),
    UNIT_PRICE NUMBER(10,2),
    REVENUE NUMBER(12,2)
);

DESCRIBE TABLE GAMING_DB.SALES.GAME_SALES;
```

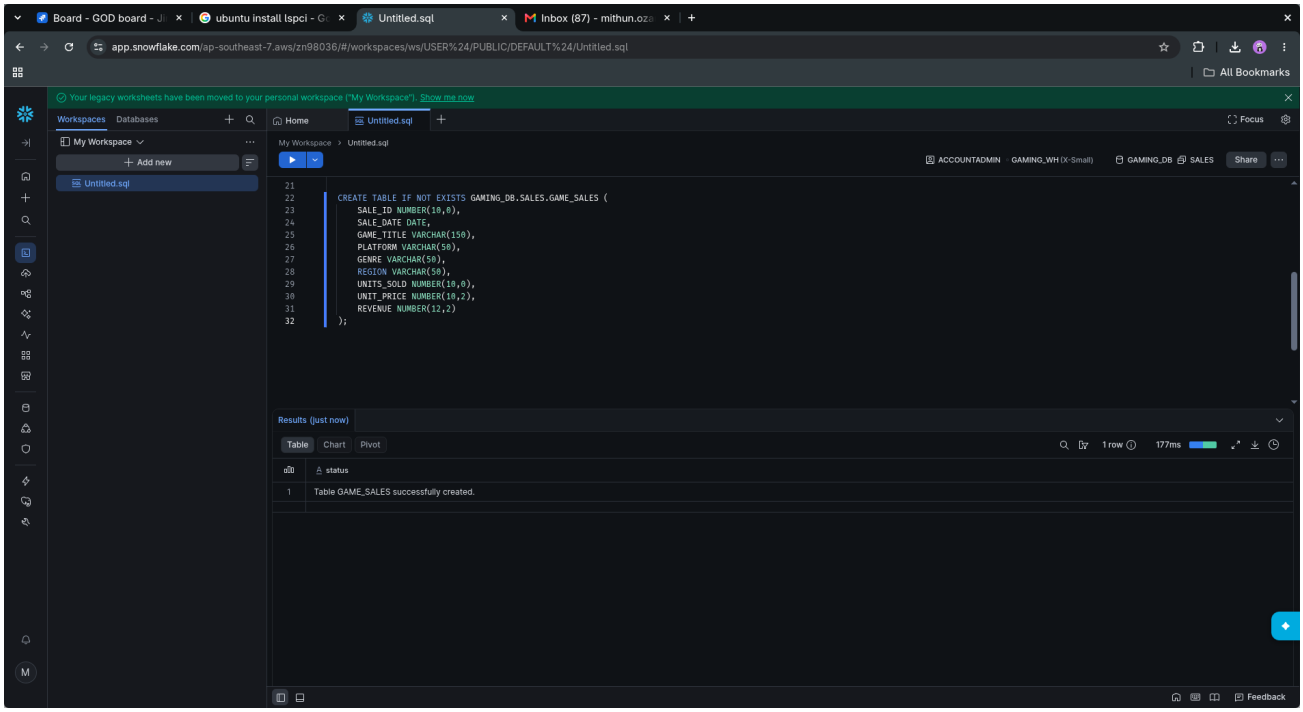


Figure 6. GAME_SALES table created.

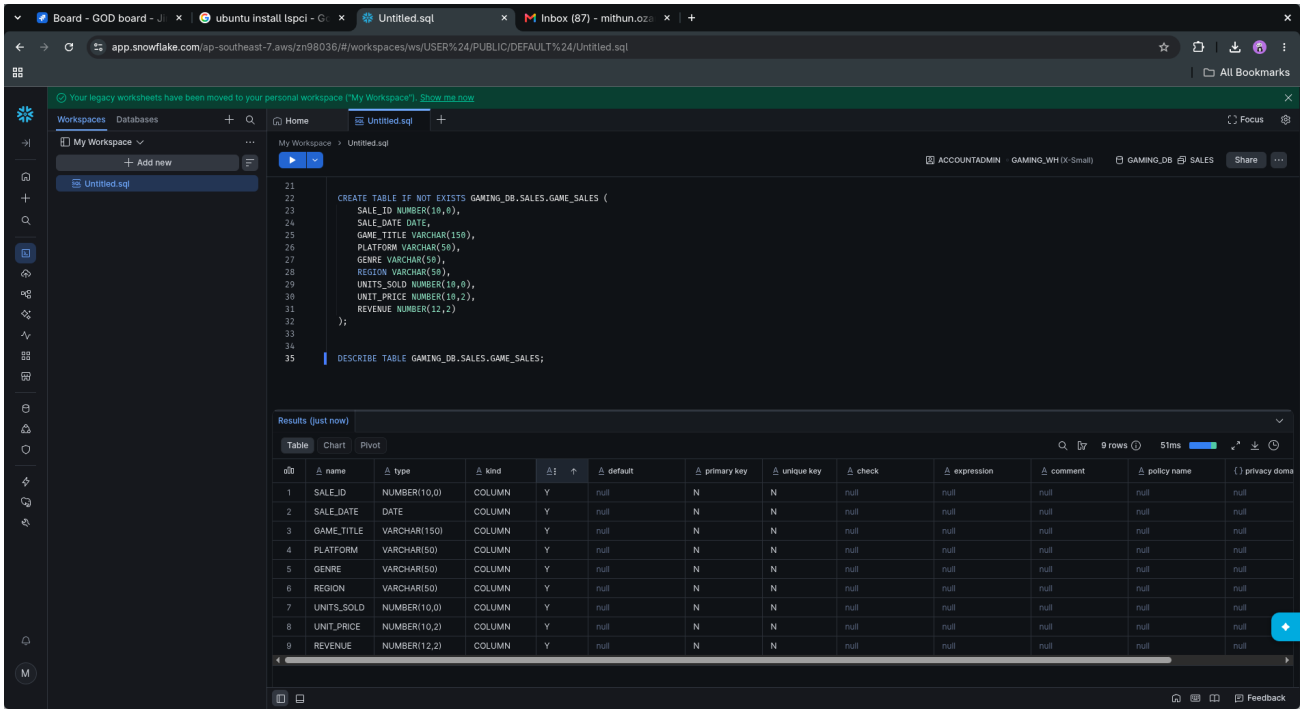


Figure 7. DESCRIBE TABLE displays the nine columns.

5. Create CSV file format

The file format specifies CSV parsing, skips the header row and handles optionally quoted and empty fields.

```
CREATE FILE FORMAT IF NOT EXISTS GAMING_DB.SALES.GAMING_CSV_FORMAT
    TYPE = CSV
    SKIP_HEADER = 1
    FIELD_OPTIONALLY_ENCLOSED_BY = '"'
    EMPTY_FIELD_AS_NULL = TRUE;

DESCRIBE FILE FORMAT GAMING_DB.SALES.GAMING_CSV_FORMAT;
```

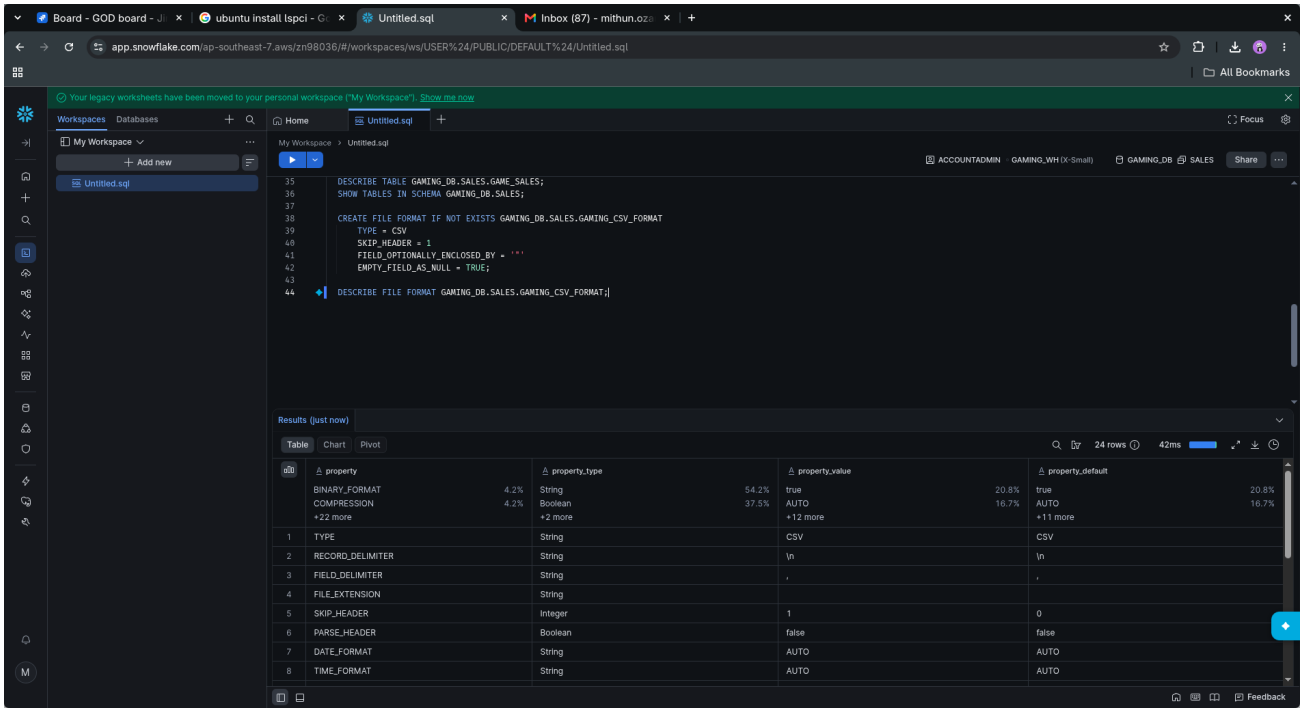


Figure 8. File format properties, including TYPE = CSV and SKIP_HEADER = 1.

6. Create internal stage

```
CREATE STAGE IF NOT EXISTS GAMING_DB.SALES.GAMING_STAGE
    FILE_FORMAT = GAMING_DB.SALES.GAMING_CSV_FORMAT;

DESCRIBE STAGE GAMING_DB.SALES.GAMING_STAGE;
LIST @GAMING_DB.SALES.GAMING_STAGE;
```

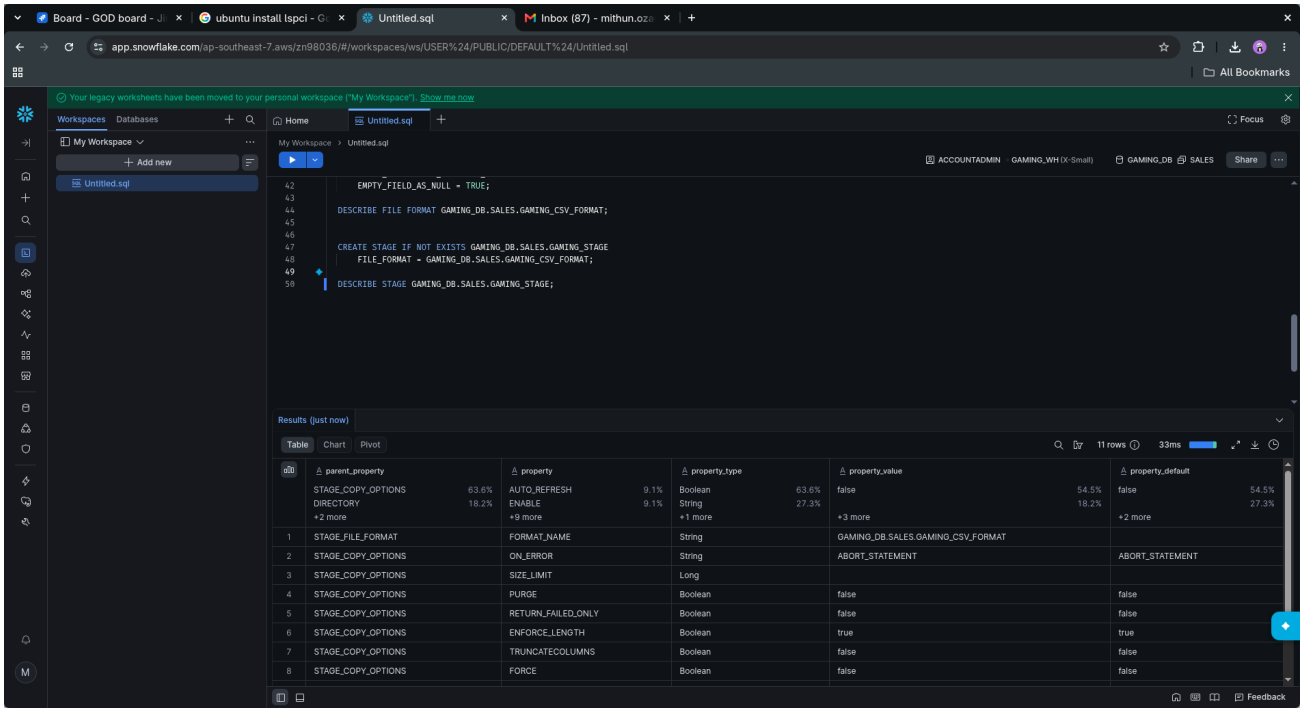


Figure 9. Stage configuration shows the associated CSV file format.

7. Upload CSV file

In Snowsight, open Catalog → Explorer → GAMING_DB → SALES → Stages → GAMING_STAGE, then select Upload Files. Upload **gaming_sales_10_records.csv**.

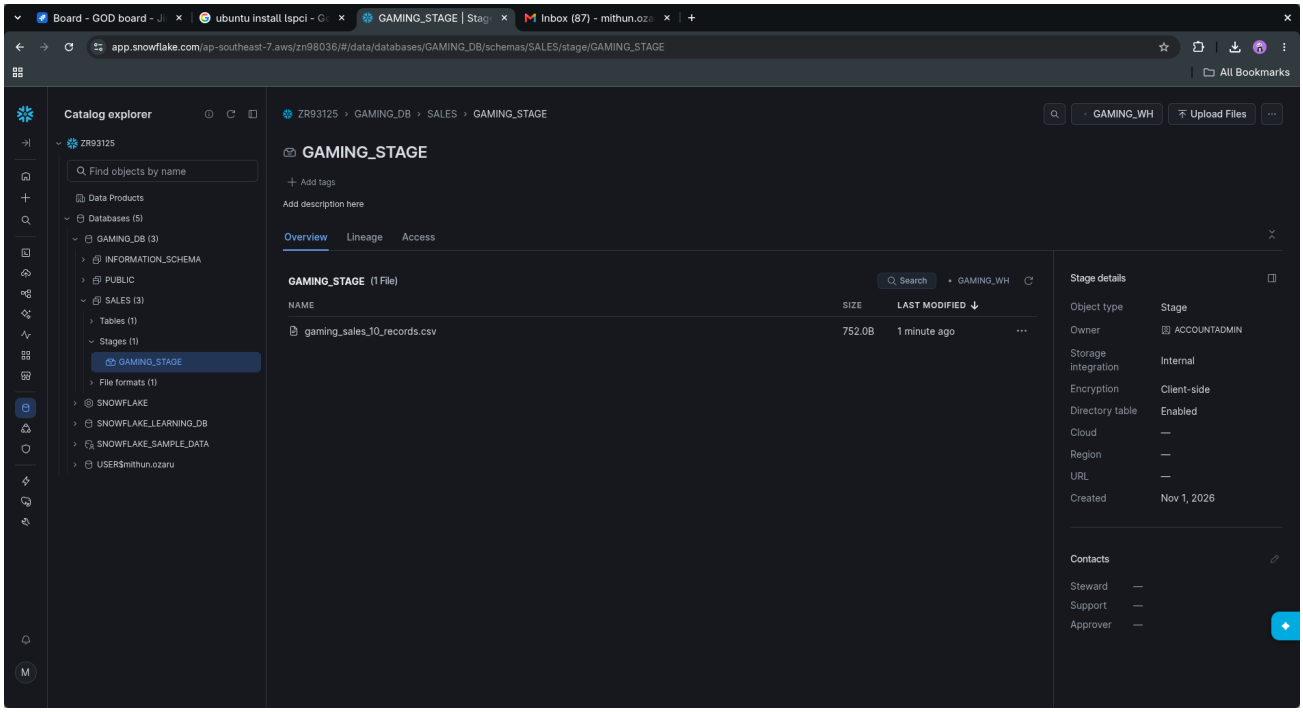


Figure 10. CSV file uploaded to GAMING_STAGE.

Verify the staged file:

```
LIST @GAMING_DB.SALES.GAMING_STAGE;
```

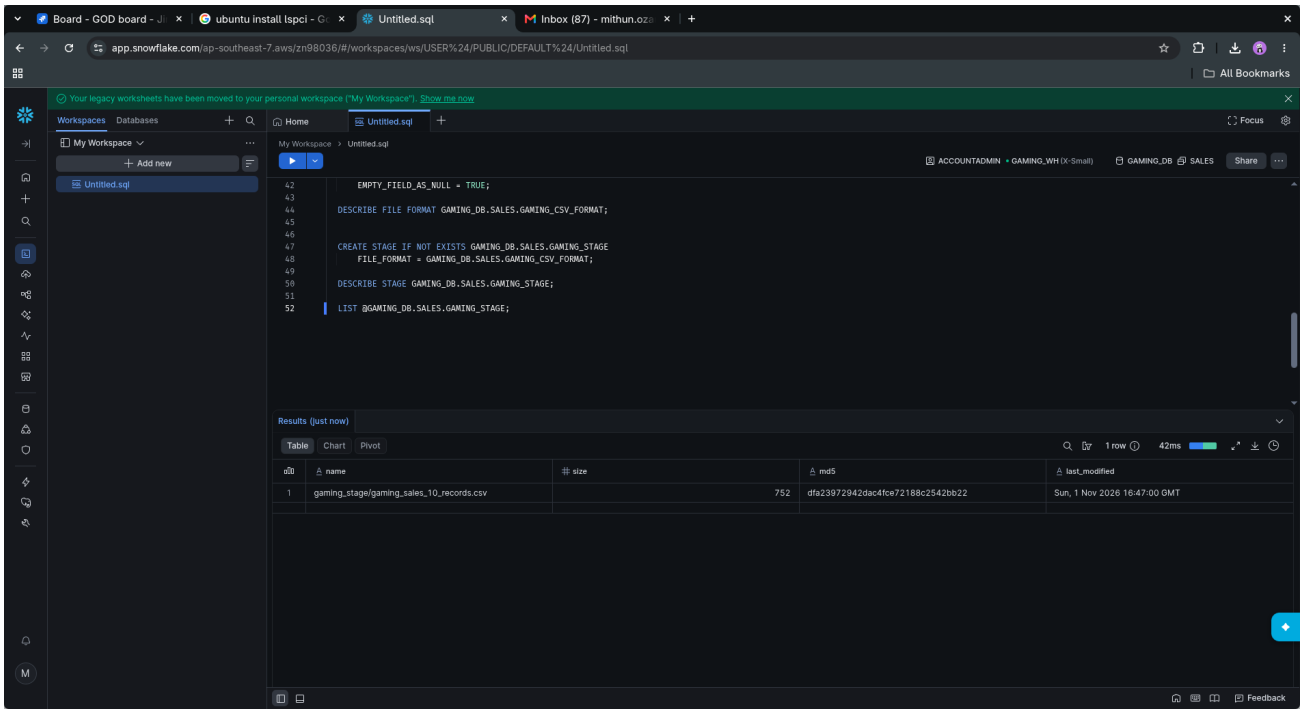


Figure 11. LIST displays the uploaded CSV file.

8. Load data using COPY INTO

Load the staged CSV into GAME_SALES. ON_ERROR = 'ABORT_STATEMENT' instructs Snowflake to abort the statement if a loading error occurs.

```
COPY INTO GAMING_DB.SALES.GAME_SALES
FROM @GAMING_DB.SALES.GAMING_STAGE
FILE_FORMAT = (
    FORMAT_NAME = 'GAMING_DB.SALES.GAMING_CSV_FORMAT'
)
ON_ERROR = 'ABORT_STATEMENT';
```

Load result: Status = LOADED; rows parsed = 10; rows loaded = 10; errors seen = 0.

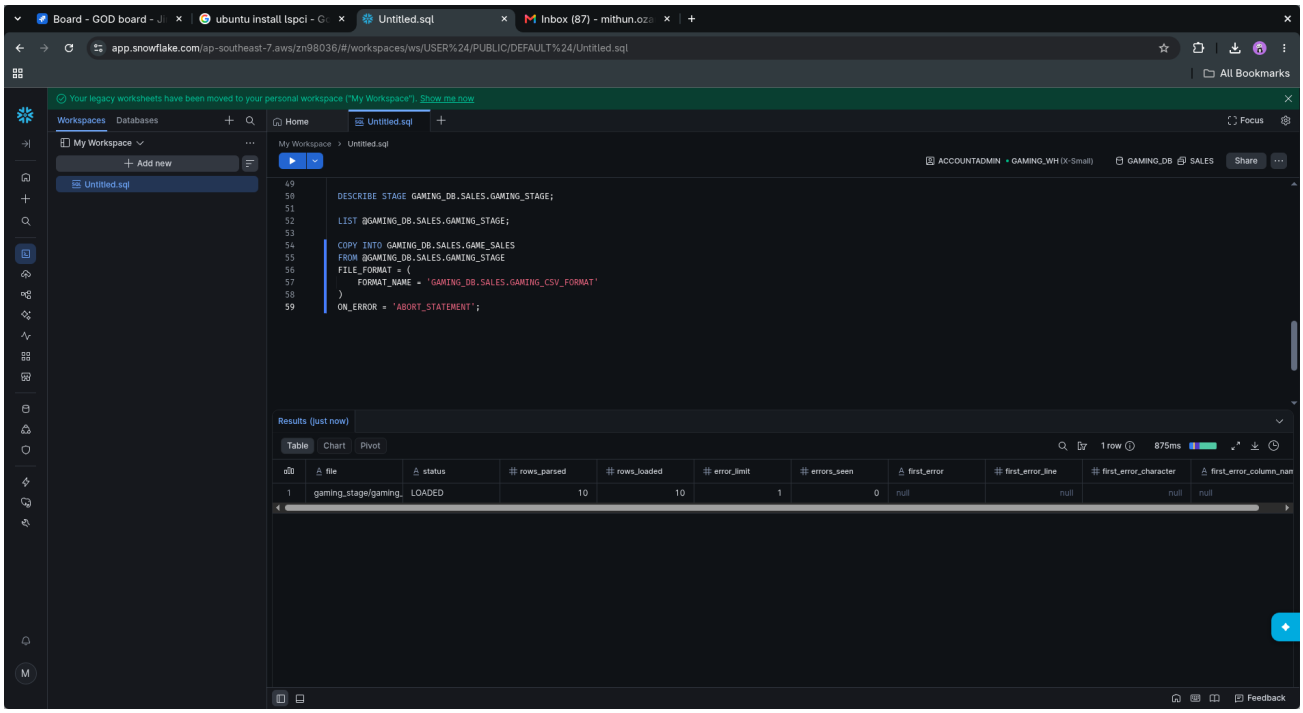


Figure 12. COPY INTO result confirms 10 rows loaded and zero errors.

9. Verify loaded records

```
SELECT *
FROM GAMING_DB.SALES.GAME_SALES;
```

The SELECT * result displayed 10 records from GAME_SALES.

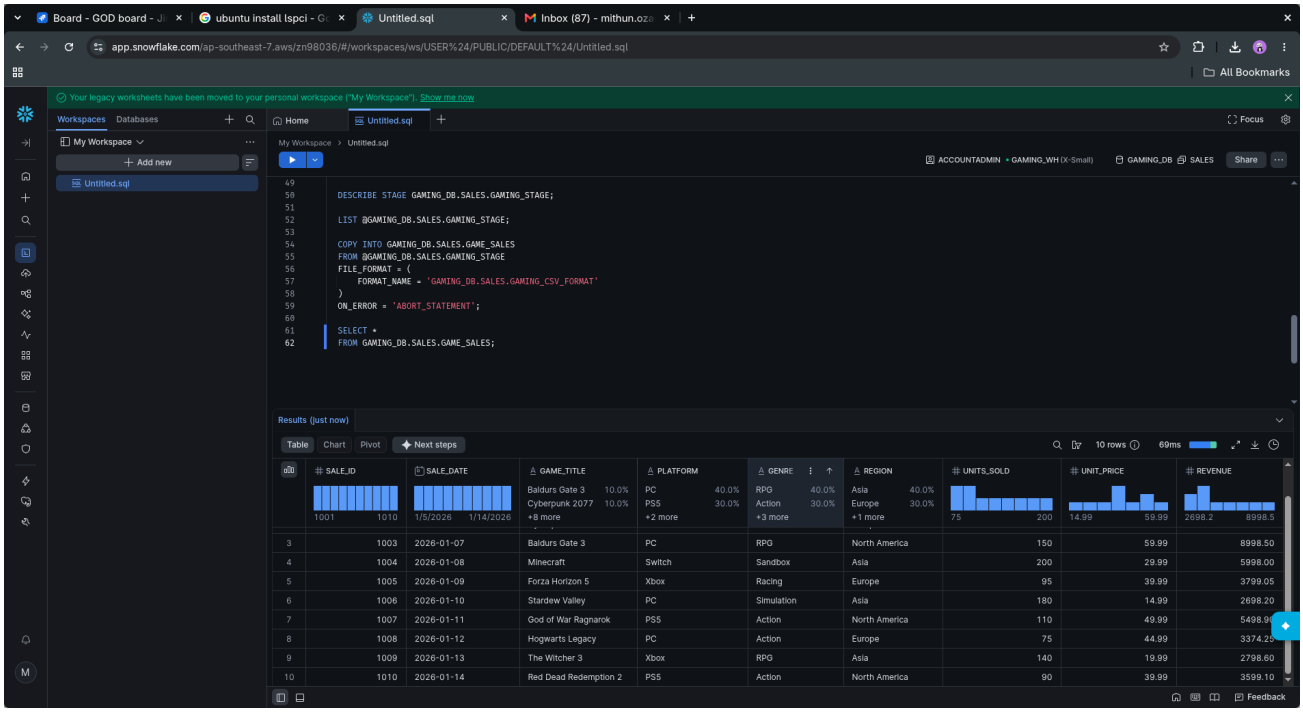


*Figure 13. SELECT * displays the loaded gaming sales records.*

10. Drop database and warehouse

```
DROP DATABASE GAMING_DB;
DROP WAREHOUSE GAMING_WH;
```

The database and warehouse were dropped after the data loading and verification steps.

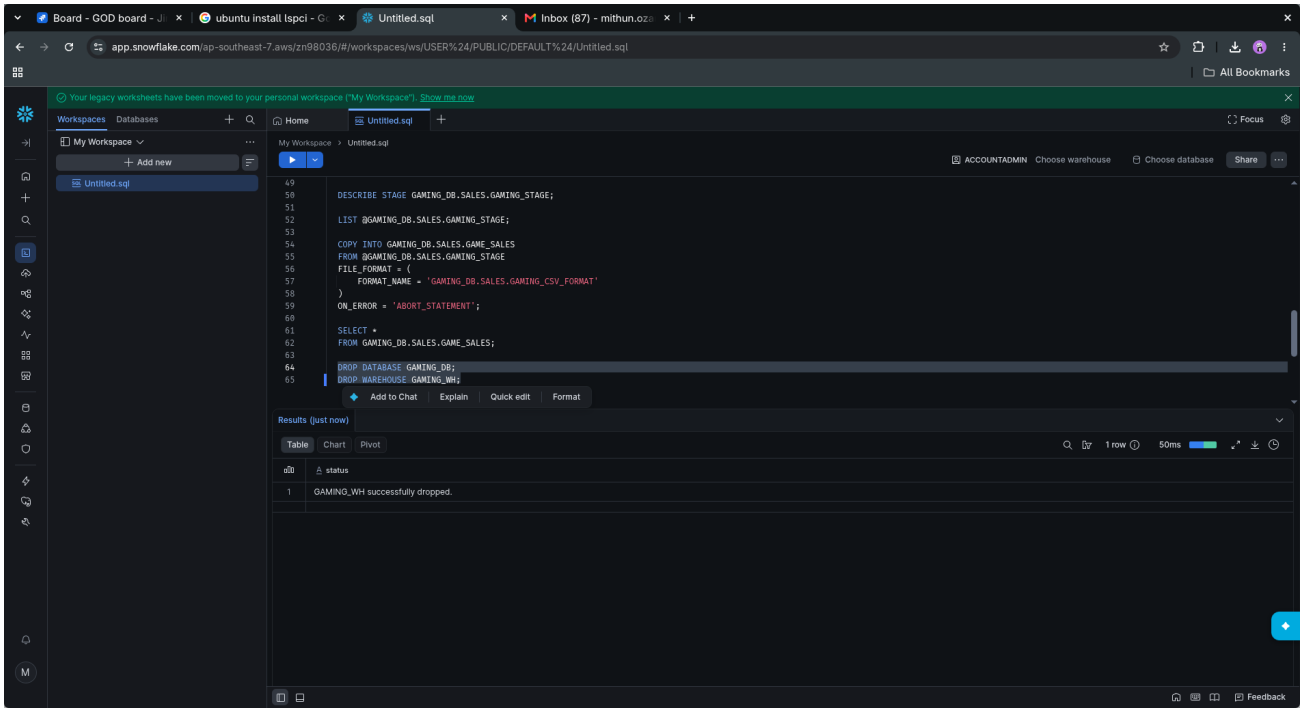


Figure 14. Final cleanup result confirms GAMING_WH was successfully dropped.

Result: The gaming sales CSV file was uploaded to a Snowflake internal stage and loaded into the GAME_SALES table. Snowflake reported 10 rows parsed, 10 rows loaded and zero errors. The loaded records were inspected using SELECT *, and the database and warehouse were dropped after completion.